
Rehosting

Secure Software & Hardware Systems

Goal of this lecture

- Introduce you to the challenges of firmware analysis
- Get you familiar with “rehosting” & “large-scale firmware analysis”
 - Give you an overview of current trends and approaches in research

Overview

- Intro
 - Challenges of firmware analysis
 - Emulation as solution?
- Rehosting
 - Hardware-in-the-loop rehosting
 - hardware-less rehosting
- Large-Scale Firmware Analysis
- Recap

Challenges of Firmware Analysis

- 1) Firmware Retrieval
- 2) Platform Variety
- 3) Fault Detection
- 4) Scalability
- 5) Instrumentation

Challenges of Firmware Analysis

~~1) Firmware Retrieval~~

2) Platform Variety

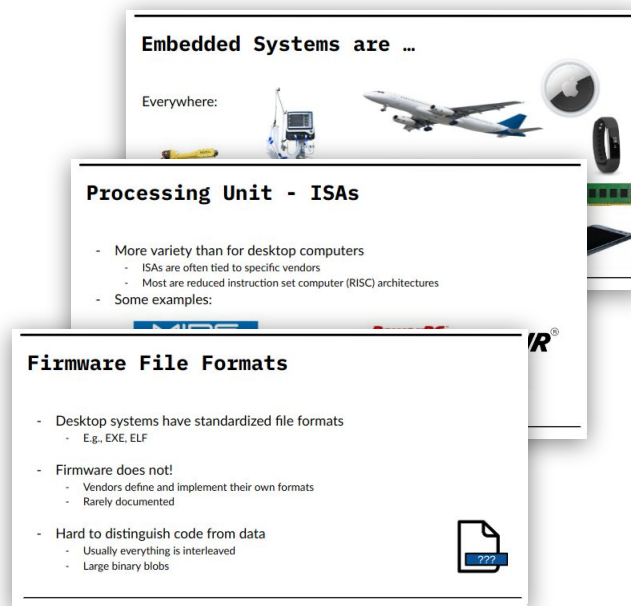
3) Fault Detection

4) Scalability

5) Instrumentation

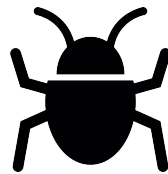
Platform Variety

- Already discussed:
 - Different ISAs
 - Firmware file formats
 - Peripherals
- Additional problems:
 - Platform configurations
 - Lack of documentation



Fault Detection

- Finding vulnerabilities often relies on observable crashes
- Desktop systems have various ways to turn corruptions into crashes, e.g.:
 - Segmentation faults
 - Stack canaries
 - Heap hardening
- These may be missing for embedded systems!

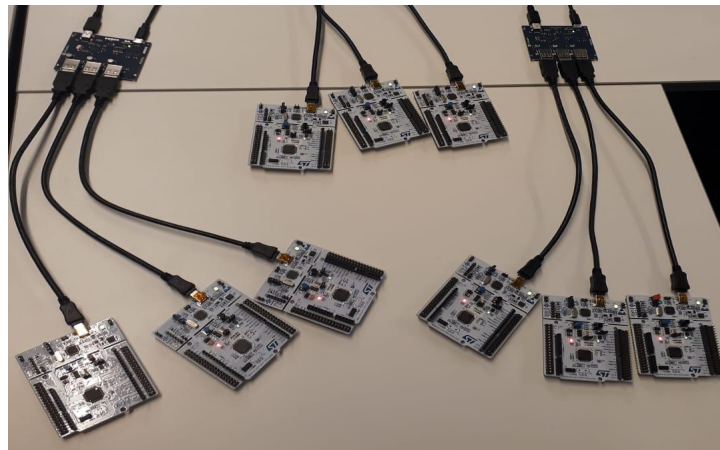


Fault Detection: Example

Platform	expat				mbed TLS			
	Desktop	Type-I	Type-II	Type-III	Desktop	Type-I	Type-II	Type-III
Format String	✓	✓	✗	✗	✓	✓	✗ (malfunc.)	! (hang)
Stack-based buffer overflow	✓	✓	✓ (opaque)	! (hang)	✓	✓	✓ (opaque)	! (hang)
Heap-based buffer overflow	✓	! (late crash)	✗	✗	✓	! (late crash)	✗	✗
Double Free	✓	✓	✗	✗ (malfunc.)	✓	! (late crash)	✗	✗
Null Pointer Dereference	✓	✓	✓ (reboot)	✗ (malfunc.)	✓	✓	✓ (reboot)	✗

Scalability

- Embedded devices are slow:
 - Low computational power
 - Long reset times/boot cycles
- But testing may rely on:
 - High execution speed
 - Fast resets
 - Multiple instances



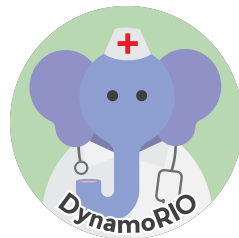
Instrumentation

- Source-based instrumentation often infeasible
- Binary instrumentation may assume:
 - Specific ISA
 - Operating system
- Further problems:
 - Missing computational resources
 - Firmware stored in ROM

FRIDA

Pin

Valgrind



Challenges of Firmware Analysis

TL;DR:

On-device analysis is problematic.

Emulation to the rescue



Emulation to the rescue



Challenges:

- ISAs
- Hardware platforms
- Firmware file formats
- Emulation accuracy

Meet: QEMU

- **Very** popular full-system emulator
 - CPU
 - Memory
 - Peripherals
- Support for a variety of ISAs
- Hardware configurations implemented in “machines”
- Also user-mode emulation support:
 - Emulation of user space software
 - System call translation layer



QEMU: Supported ARM boards

```
nsr@xpad ~ % qemu-system-arm --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.10)
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project developers
nsr@xpad ~ % qemu-system-arm -M help
Supported machines are:
akita          Sharp SL-C1000 (Akita) PDA (PXA270)
ast1030-evb    Aspeed AST1030 MiniBMC (Cortex-M4)
ast2500-evb    Aspeed AST2500 EVB (ARM1176)
ast2600-evb    Aspeed AST2600 EVB (Cortex-A7)
bletchley-bmc  Facebook Bletchley BMC (Cortex-A7)
borzoi         Sharp SL-C3100 (Borzoi) PDA (PXA270)
bpim2u         Bananapi M2U (Cortex-A7)
canon-all100  Canon PowerShot A1100 IS (ARM946)
cheetah        PaLM Tungsten|E aka. Cheetah PDA (OMAP310)
collie         Sharp SL-5500 (Collie) PDA (SA-1110)
connex         Gumstix Connex (PXA255)
cubieboard     Cubietech Cubieboard (Cortex-A8)
emcraft-sf2    SmartFusion2 SOM kit from Emcraft (M2S010)
fbys35-bmc     Facebook fby35 BMC (Cortex-A7)
fbys35         Meta Platforms fby35
fp5280g2-bmc   Inspur FP5280G2 BMC (ARM1176)
fuji-bmc       Facebook Fuji BMC (Cortex-A7)
g220a-bmc      Bytedance G220A BMC (ARM1176)
highbank       Calxeda Highbank (ECX-1000)
imx25-pdk      ARM i.MX25 PDK board (ARM926)
integratorcp   ARM Integrator/CP (ARM926J-S)
kudo-bmc       Kudo BMC (Cortex-A9)
kzm           ARM KZM Emulation Baseboard (ARM1136)
lm3s6965evb    Stellaris LM3S6965EVB (Cortex-M3)
lm3s811evb     Stellaris LM3S811EVB (Cortex-M3)
mainstone      Mainstone II (PXA27x)
mcimx6ul-evk   Freescale i.MX6UL Evaluation Kit (Cortex-A7)
mcimx7d-sabre  Freescale i.MX7 DUAL SABRE (Cortex-A7)
microbit       BBC micro:bit (Cortex-M0)
midway         Calxeda Midway (ECX-2000)
mori-bmc       Mori BMC (Cortex-A9)
mps2-an385     ARM MPS2 with AN385 FPGA image for Cortex-M3
mps2-an386     ARM MPS2 with AN386 FPGA image for Cortex-M4
mps2-an500     ARM MPS2 with AN500 FPGA image for Cortex-M7
mps2-an505     ARM MPS2 with AN505 FPGA image for Cortex-M33
mps2-an511     ARM MPS2 with AN511 DesignStart FPGA image for Cortex-M3
mps2-an521     ARM MPS2 with AN521 FPGA image for dual Cortex-M33
mps3-an524     ARM MPS3 with AN524 FPGA image for dual Cortex-M33
mps3-an547     ARM MPS3 with AN547 FPGA image for Cortex-M55
```

```
musca-a        ARM Musca-A board (dual Cortex-M33)
musca-b1       ARM Musca-B1 board (dual Cortex-M33)
musicpal       Marvell 88w8618 / MusicPal (ARM926EJ-S)
n800           Nokia N800 tablet aka. RX-34 (OMAP2420)
n810           Nokia N810 tablet aka. RX-44 (OMAP2420)
netduino2      Netduino 2 Machine (Cortex-M3)
netduinoplus2 Netduino Plus 2 Machine (Cortex-M4)
none          empty machine
npcm750-evb    Nuvoton NPCM750 Evaluation Board (Cortex-A9)
nuri           Samsung NURI board (Exynos4210)
olimex-stm32-h405 Olimex STM32-H405 (Cortex-M4)
orangepi-pc    Orange Pi PC (Cortex-A7)
palmetto-bmc   OpenPOWER Palmetto BMC (ARM926EJ-S)
qcom-dc-scm-v1-bmc Qualcomm DC-SCM V1 BMC (Cortex-A7)
qcom-firework-bmc Qualcomm DC-SCM V1/Firework BMC (Cortex A7)
quanta-gbs-bmc Quanta GBS (Cortex-A9)
quanta-gsj     Quanta GSJ (Cortex-A9)
quanta-q71l-bmc Quanta-Q71L BMC (ARM926EJ-S)
rainier-bmc    IBM Rainier BMC (Cortex-A7)
raspi0         Raspberry Pi Zero (revision 1.2)
raspi0b        Raspberry Pi A+ (revision 1.1)
raspi2b        Raspberry Pi 2B (revision 1.1)
realview-eb    ARM RealView Emulation Baseboard (ARM926EJ-S)
realview-eb-mpcore ARM RealView Emulation Baseboard (ARM11MPCore)
realview-pb-a8 ARM RealView Platform Baseboard for Cortex-A8
realview-pbx-a9 ARM RealView Platform Baseboard Explore for Cortex-A9
romulus-bmc    OpenPOWER Romulus BMC (ARM1176)
sabrelite      Freescale i.MX6 Quad SABRE Lite Board (Cortex-A9)
smdkc210       Samsung SMDKC210 board (Exynos4210)
sonorapass-bmc OCP SonoraPass BMC (ARM1176)
spitz          Sharp SL-C3000 (Spitz) PDA (PXA270)
stm32vldiscovery ST STM32VLDISCOVERY (Cortex-M3)
supermicro-xl1spi-bmc Supermicro X11 SPI BMC (ARM1176)
supermicrox1l-bmc Supermicro X11 BMC (ARM926EJ-S)
sx1            Siemens SX1 (OMAP310) V2
sx1-v1         Siemens SX1 (OMAP310) V1
tacoma-bmc     OpenPOWER Tacoma BMC (Cortex-A7)
terrier        Sharp SL-C3200 (Terrier) PDA (PXA270)
tiogapass-bmc  Facebook Tiogapass BMC (ARM1176)
tosa           Sharp SL-6000 (Tosa) PDA (PXA255)
verdex         Gumstix Verdex Pro XL6P COMs (PXA270)
versatileab    ARM Versatile/AB (ARM926EJ-S)
versatilepb    ARM Versatile/PB (ARM926EJ-S)
vexpress-a15   ARM Versatile Express for Cortex-A15
```

```
virt-2.10      QEMU 2.10 ARM Virtual Machine
virt-2.11      QEMU 2.11 ARM Virtual Machine
virt-2.12      QEMU 2.12 ARM Virtual Machine
virt-2.6       QEMU 2.6 ARM Virtual Machine
virt-2.7       QEMU 2.7 ARM Virtual Machine
virt-2.8       QEMU 2.8 ARM Virtual Machine
virt-2.9       QEMU 2.9 ARM Virtual Machine
virt-3.0       QEMU 3.0 ARM Virtual Machine
virt-3.1       QEMU 3.1 ARM Virtual Machine
virt-4.0       QEMU 4.0 ARM Virtual Machine
virt-4.1       QEMU 4.1 ARM Virtual Machine
virt-4.2       QEMU 4.2 ARM Virtual Machine
virt-5.0       QEMU 5.0 ARM Virtual Machine
virt-5.1       QEMU 5.1 ARM Virtual Machine
virt-5.2       QEMU 5.2 ARM Virtual Machine
virt-6.0       QEMU 6.0 ARM Virtual Machine
virt-6.1       QEMU 6.1 ARM Virtual Machine
virt-6.2       QEMU 6.2 ARM Virtual Machine
virt-7.0       QEMU 7.0 ARM Virtual Machine
virt-7.1       QEMU 7.1 ARM Virtual Machine
virt-7.2       QEMU 7.2 ARM Virtual Machine
virt-8.0       QEMU 8.0 ARM Virtual Machine
virt-8.1       QEMU 8.1 ARM Virtual Machine
virt           QEMU 8.2 ARM Virtual Machine (alias of virt-8.2)
virt-8.2       QEMU 8.2 ARM Virtual Machine
witherspoon-bmc OpenPOWER Witherspoon BMC (ARM1176)
xilinx-zynq-a9 Xilinx Zynq Platform Baseboard for Cortex-A9
yosemitev2-bmc Facebook YosemiteV2 BMC (ARM1176)
z2             Zipit Z2 (PXA27x)
```

Boards out there: Reality

- Cortex-M chips only
- For one vendor
- With documentation

Overview

- Intro
 - Challenges of firmware analysis
 - Emulation as solution?
- Rehosting
 - Hardware-in-the-loop rehosting
 - hardware-less rehosting
- Large-Scale Firmware Analysis
- Recap

Rehosting

“Firmware re-hosting is the process of migrating firmware from its original hardware environment into a **virtual environment**”

Rehosting vs Emulation

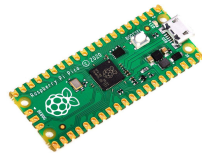
Rehosting:

- Firmware-centric
- Make firmware analyzable
- Recently emerged research topic

Emulation:

- Hardware-centric
- Faithful mimicking of hardware platform
- Around since the last century

Meet: Unicorn



- QEMU-based CPU emulation framework
 - Removed code for memory and peripherals
- Various language bindings:
 - Rust
 - Python
 - ...
- Allows quickly prototyping for rehosting small parts of a firmware



Meet: avatar2

- Python-based multi-target orchestration system
- Supports orchestration & combination of:
 - QEMU, Unicorn, OpenOCD, GDB, and more!
- Focus on firmware analysis & rehosting
 - Extends QEMU with a “configurable” machine
 - Remote memory & state transfer
 - Python peripheral models

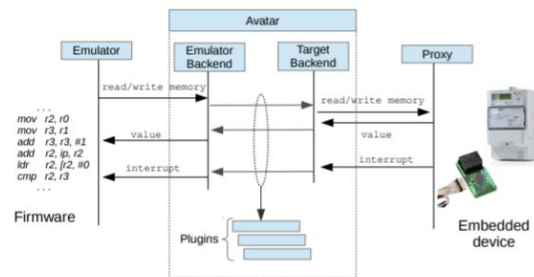


Hardware-In-The-Loop Rehosting

- Also known as partial emulation
- Key ideas:
 - Avoid emulation of complex/unknown HW
 - Instead: forward hardware interaction to device
- Usually requires debugging ports or stubs

Avatar

- Early HITL frameworks with focus on security analysis
 - Based on S2E (Symbolic execution platform on top of QEMU)
 - Requires JTAG or injectable debug stub
- Important concepts:
 - Remote memory
 - Separation of execution and memory
 - Forwarding of peripheral interaction
- State transfer:
 - Remote memory is slow
 - Execute code with heavy I/O on device, then switch to emulator
- Configurable machine
 - A “machine” for QEMU assembled on-the-fly based on json config file



Surrogates

- High-performant HITL framework
 - Based on QEMU & PCIe FPGA + custom JTAG debugger
- Eliminates performance issues of avatar
 - Fast enough forwarding to boot Linux
- Requires very specialized hardware
 - Used FPGA is not produced anymore

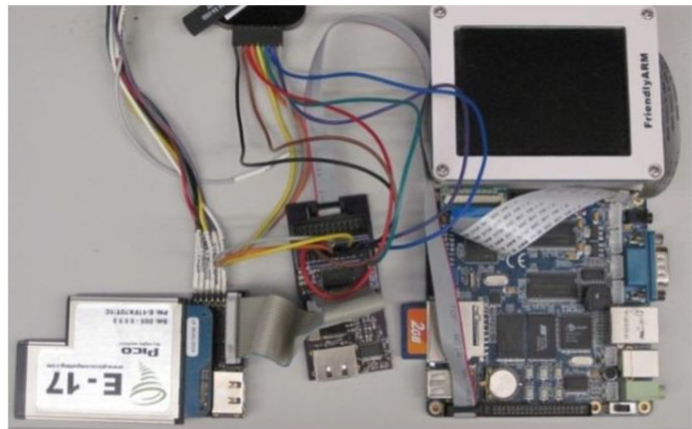


Image source & more information:

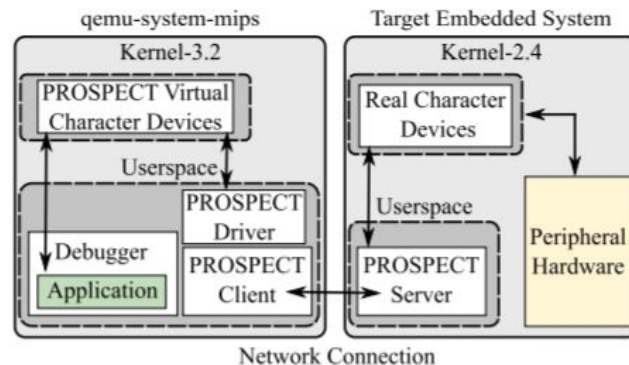
K. Koscher et al. "SURROGATES: Enabling Near-Real-Time Dynamic Analyses of Embedded Systems." WOOT'15

Prospect

- Early HITL Rehosting for Linux-based systems
 - Based on QEMU & custom device drivers + on target stub

Base assumption:

- HW is abstracted by special character devices
 - Only forward accesses to these devices
 - Only userspace-stub injected on target
 - Special handling of IOCTL



Summary: HITL Rehosting

- Just a couple of approaches inspected
 - There are more
- Scalability is still limited
 - Device dependency
 - Multiplexing possible, but impractical

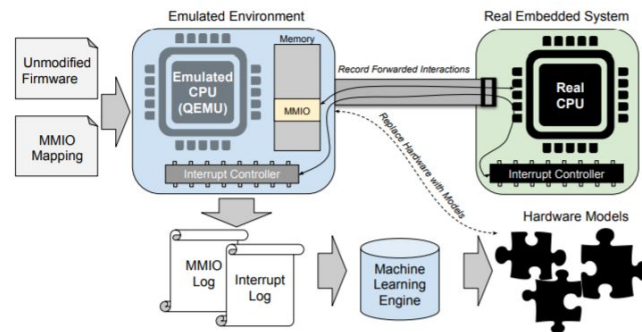
	Avatar	Surrogates	Prospect
Firmware Type	2-3	1-3	1
High Performance Impact	Yes	No	Yes
Debug Port Required	Yes	Yes	No
Open Source	Yes	No	No

Hardware-less Rehosting

- Also known as full emulation
- Key ideas:
 - (Semi-)automatically create HW models
 - Completely eliminate device dependency
- Higher likelihood for inaccuracies
 - Inaccurate models
 - Ground truth may be missing

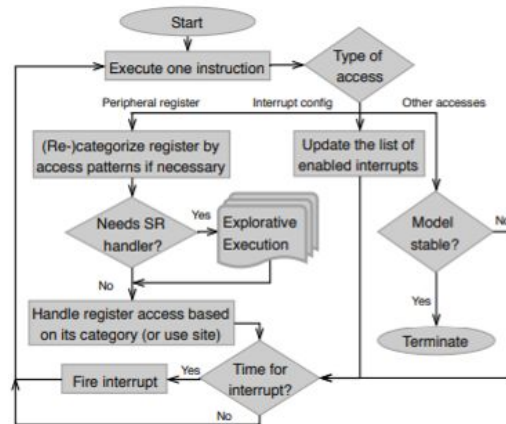
Pretender

- HITL scheme for enabling hardware-less rehosting
 - Based on avatar2
- HITL rehosting only during learning phase
 - Record peripheral interaction
 - Analyze records & derive models:
 - Simple Storage
 - Pattern
 - Increasing
 - Write-Only
 - Fall-back: stateful replay
- Use hardware models for hardware-less rehosting



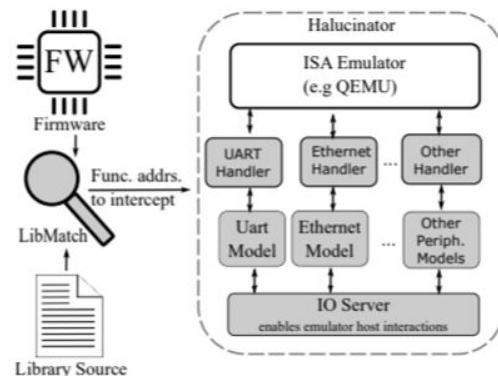
P2IM

- Hardware-less rehosting for Type-2/3 firmware with automated peripheral model creation
 - Based on QEMU
- Uses heuristics to identify register types:
 - Control Registers
 - Status Registers
 - Data Registers
 - Control-Status Registers
- Instantiate model based on register type



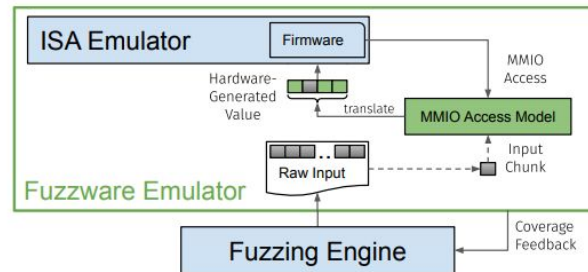
HALucinator

- Different approach to hardware-less rehosting!
 - Based on avatar2 & Unicorn
- Key observation: HW accesses typically go through a Hardware Abstraction Layers (HAL)
- Automatically identify HAL functions, e.g.:
 - HAL_TIM_OscConfig
 - HAL_Ethernet_RX_Frame
- Provide handlers for identified functions:
 - Often similar across different HALs
 - Also known as “High-Level Emulation”



Fuzzware

- Points out limitations of rehosting approaches for fuzzing
 - Path elimination
 - Input overhead
- Rehhosts from a fuzzing first perspective
 - Every MMIO access is seen as fuzzing input
 - Uses dynamic symbolic execution to reduce input space
 - Based on Unicorn



Firmadyne

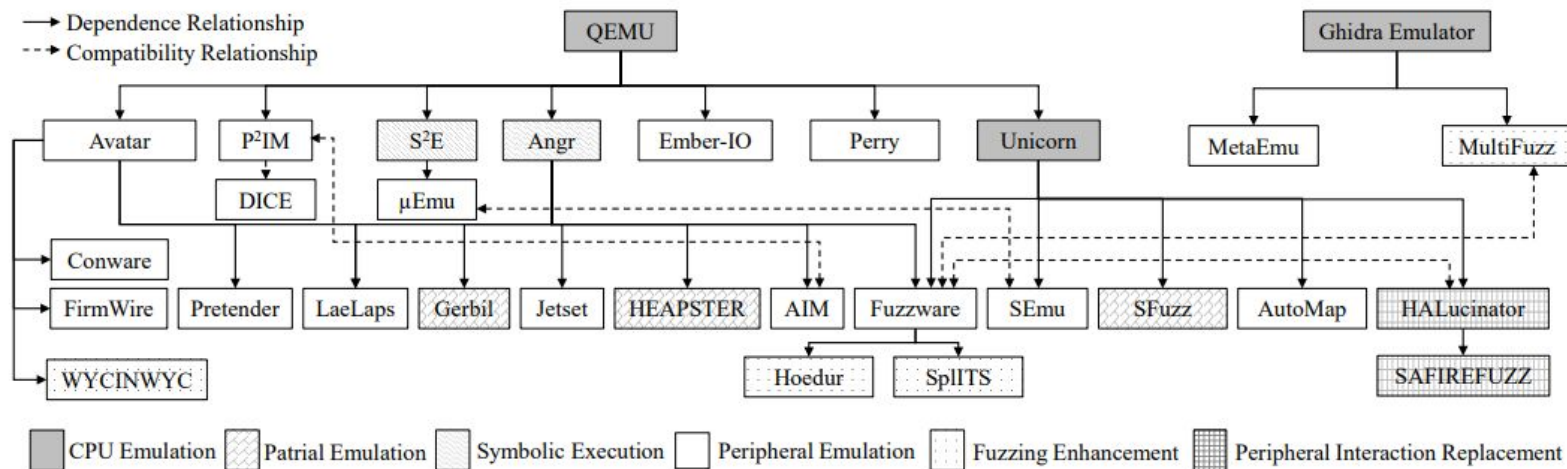
- Hardware-less rehosting for Linux-based systems
 - Based on QEMU
- Unpacks Linux-based firmware and extract file-system
 - Emulation in QEMU with custom kernel
 - Provide custom library for NVRAM accesses
 - Adhoc modifications in QEMU's flash device implementation
- Evaluation on 23,035 firmware images

Summary: Hardware-less Rehosting

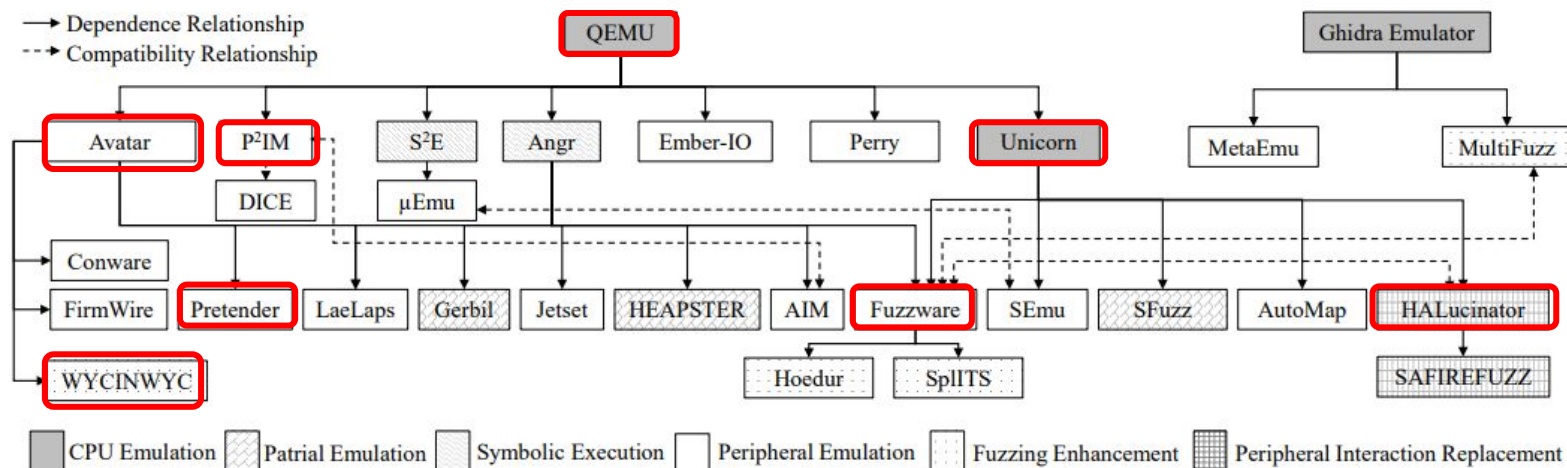
- Most hardware-less rehosting for type-2/3 systems focuses on register modeling
- QEMU is the single-most prominent emulator for rehosting research

	Pretender	P2IM	HALucinator	Fuzzware	Firmadyne
Firmware Type	2-3	2-3	2-3	3	1
Rehosting Approach	Register Modelling	Register Modelling	High-Level Emulation	Register Modelling	OS abstractions
Device Required	Yes	No	No	No	No
Open Source	Yes	Yes	Yes	Yes	Yes

Rehosting: A State of the Art



Rehosting: A State of the Art



Overview

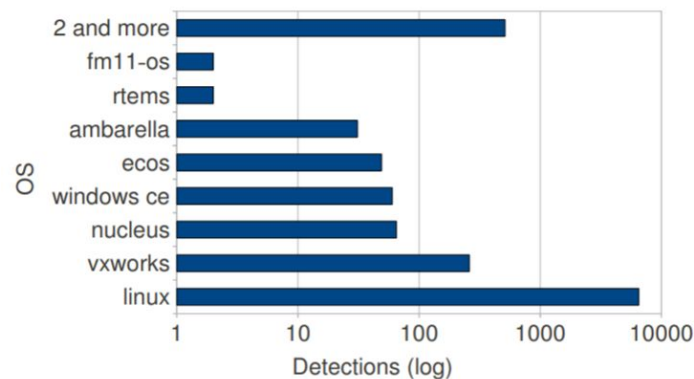
- Intro
 - Challenges of firmware analysis
 - Emulation as solution?
- Rehosting
 - Hardware-in-the-loop rehosting
 - hardware-less rehosting
- Large-Scale Firmware Analysis
- Recap

Large-Scale Analysis

- “Shotgun” approach to firmware analysis
 - Different set of analysis techniques
 - Increased logic for retrieval and unpacking
- Generates insights about the current state of embedded systems
 - At least for systems with crawlable updates
 - In practice: Routers, IP cameras, ...

Large-Scale Analysis '14

- “First public, large-scale analysis of firmware images” (sic)
 - ~32000 firmware images
 - Unpacking based on the Binary Analysis Toolkit (BAT)
 - Purely static analysis
- Identified problems:
 - Hardcoded passwords
 - Private certificates included in firmware updated
 - Backdoors in plain sight (authorized_keys)



OS distribution in dataset

Firmadyne

- Automated Dynamic Analysis
 - ~23000 firmware images
 - Unpacking based on binwalk
 - Emulation/Rehosting-based approach
- Identified problems:
 - Known exploits are often reusable
 - Even across firmware from different vendors
 - Hints towards code-reuse!

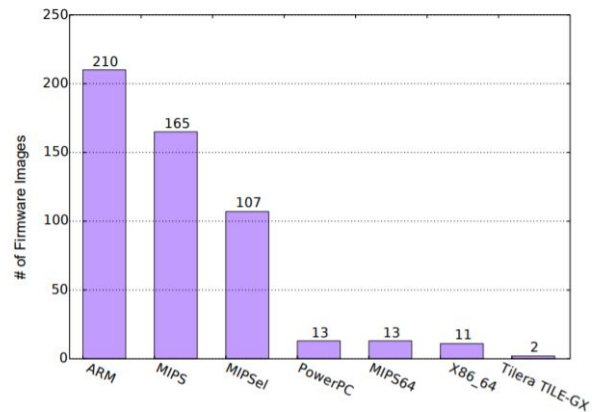
Architecture (Endian)	# Image(s)
TILE (LE)	1
ARC (LE)	10
Motorola 68k (BE)	10
x86 (LE)	31
MIPS 64-bit (BE)	50
PPC (BE)	84
ARM (BE)	102
x86-64 (LE)	147
Unknown	439
ARM (LE)	843
MIPS (BE)	3,137
MIPS (LE)	4,632
Total	9,486

Type	# Images
Linux	9,379
Unidentified (UNIX-like)	2,187
VxWorks	857
Unknown	10,612
Total	23,035

**ISA and OS distribution
in Firmadyne dataset**

CryptoRex

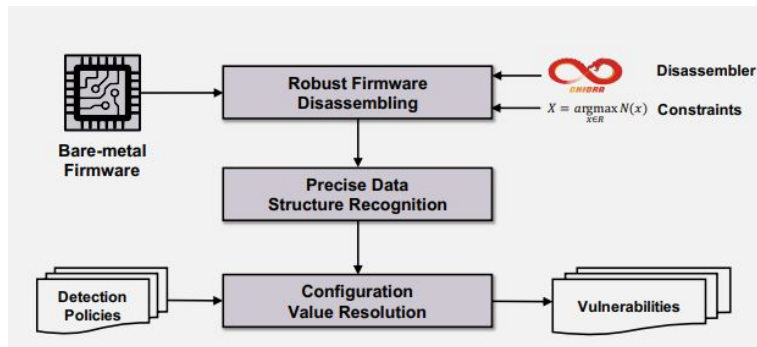
- Detecting cryptographic mistakes in firmware
 - Dataset of ~1300 firmware images
 - Crawling based on Firmadyne
 - Unpacking based on binwalk
 - Try to discover violation of cryptographic “rules”
 - Analysis on taint analysis with angr
- Identified problem:
 - Insecure usage of crypto libraries is common



ISA distribution in CryptoREX dataset

FirmXRay

- Targets Bluetooth Low Energy (BLE) firmware
 - Data set of ~800 samples
 - Extraction of firmware from APKs
 - Automated base address recognition for robust disassembly
 - Vulnerability detection via policies
- Identified problem:
 - Insecure usage of BLE is very common for bare-metal IoT devices



Summary: Large-Scale Analysis

- Most large-scale analysis is focused on Linux-based devices
 - Although plenty of type-2 firmware found!

	Costin'14	Firmadyne	CryptoREX	FirmXRay
Firmware Type	1	1	1	2-3
Analysis Approach	Static Analysis	Rehosting	Taint Analysis	Static Analysis
Unpacking	BAT	Binwalk	Binwalk	Custom
Sample Size	~32000	~23000	~500	~800
Open Source	No	Yes	Yes	Yes

Overview

- Intro
 - Challenges of firmware analysis
 - Emulation as solution?
- Rehosting
 - Hardware-in-the-loop rehosting
 - hardware-less rehosting
- Large-Scale Firmware Analysis
- Recap

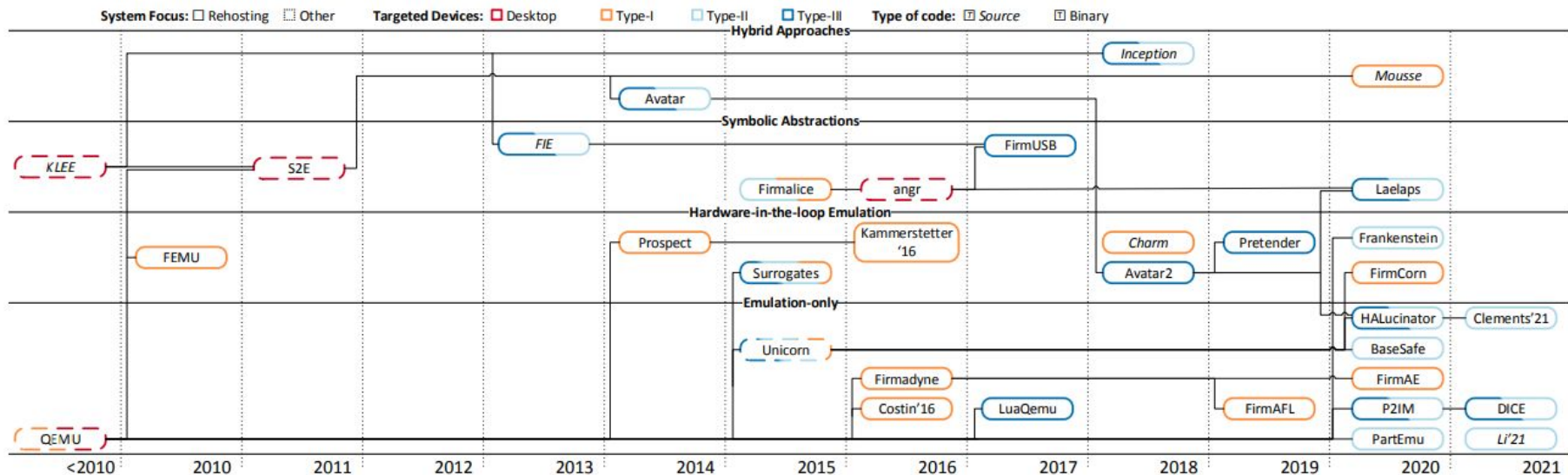
Recap

- The difficulties of firmware analysis
 - “Why” is on-device analysis not enough?
- Rehosting
 - “How” can we effectively analyze specific firmware?
- Large-Scale Analysis
 - “How” can we effectively analyze a lot of firmware?
 - “What” are the lessons learned from prior analyses?

Rehosting: Skipped Topics & Further Reading

- Interrupt Handling
 - B. Feng et al. "Aim: Automatic interrupt modeling for dynamic firmware analysis." IEEE TDSC'23
- Direct Memory Accesses (DMA)
 - A. Mera et al. "DICE: Automatic Emulation of DMA Input Channels for Dynamic Firmware Analysis." IEEE S&P'21
 - T. Scharnowski et al. "GDMA: Fully Automated DMA Rehosting via Iterative Type Overlays." USENIX Sec'25
- Symbolic Abstractions for Rehosting
 - D. Davidson et al. "FIE on firmware: Finding vulnerabilities in embedded systems using symbolic execution." USENIX Sec'13
 - G. Hernandez et al. "FirmUSB: Vetting USB Device Firmware using Domain Informed Symbolic Execution." CCS'17
 - N. Corteggiani et al. "Inception: System-wide security testing of real-world embedded systems software." USENIX Sec'18
 - W. Zhou et al. "Automatic Firmware Emulation through Invalidity-guided Knowledge Inference." USENIX Sec'21
 - E. Johnson et al. "Jetset: Targeted Firmware Rehosting for Embedded Systems." USENIX Sec'21

Rehosting: A Historical Perspective



Rehosting: A Historical Perspective

